

# MedAssist AI

An AI-Assisted Symptom Intelligence, Risk Assessment & Treatment Recommendation Platform

**4**

Model  
components

**~900K**

Training  
records

**10**

Chronic risk  
conditions

**2-layer**

Treatment  
cascade

## PROJECT DOCUMENTATION

Prepared by

**Dharani K**

September 2026

---

Reference implementation: MedAssist AI (Milestone 2 and onward)

|             |                                                                                                          |
|-------------|----------------------------------------------------------------------------------------------------------|
| Project     | MedAssist AI                                                                                             |
| Prepared by | Dharani K                                                                                                |
| Version     | 1.0                                                                                                      |
| Status      | Phases 1-3 complete; Phase 4 (treatment cascade) integrated, gate re-tuning open — see Sections 8 and 11 |
| Date        | September 2026                                                                                           |

---

## 1. Project Overview & Objectives

MedAssist AI takes a patient through one connected pipeline: reported symptoms become a ranked set of likely conditions, those conditions combine with a lifestyle profile to estimate chronic disease risk, the combined picture is scored for urgency by a transparent rule engine, and — where a condition is confirmed — the system surfaces treatment patterns from real clinical prescribing data, falling back to patient-reported outcomes. The goal is to compress the time between "something feels wrong" and "here is what to do next", with every recommendation traceable back to the evidence behind it.

The platform is built around four cooperating components rather than one monolithic model: a disease classifier, a chronic-risk scorer, a rule-based severity engine, and a two-layer treatment recommender. Each is trained and evaluated independently, then composed into a single workflow — keeping every piece auditable on its own terms, and letting one layer improve (e.g. a new evidence source for treatment) without retraining the rest.

### 1.1 Objectives

- Predict the most probable diseases from a set of reported symptoms, ranked with a calibrated confidence rather than a single forced answer.
- Estimate population-relative risk for ten major chronic conditions (diabetes, heart attack, coronary heart disease, stroke, asthma, arthritis, depression, kidney disease, skin cancer, and other cancers) from a lifestyle and demographic profile.
- Escalate genuinely dangerous presentations to an **Emergency** triage level immediately, using auditable, clinician-defined rules rather than a learned model, because no labelled triage dataset exists to train one safely.
- Recommend treatment patterns for a confirmed or suspected condition, preferring real hospital prescribing evidence over patient-reported drug ratings whenever the evidence supports it, and falling back transparently when it does not.
- Give both patients and healthcare providers a dashboard where every score, ranking, and recommendation is accompanied by the evidence and caveats behind it — never a bare number.
- Keep the system honest about its own limits: every model in the pipeline is benchmarked against a deliberately dumb baseline, and every treatment answer states which evidence source produced it and why.

### 1.2 Who this is for

Two user types are served directly: **patients**, who use the symptom checker and risk assessment to understand what they are experiencing and how urgently they should seek care; and **healthcare providers**, who use the same underlying assessments alongside a provider reporting view to review a patient's history, add clinical insights, and issue prescriptions within the platform. The system is explicitly positioned as **decision support**, not a diagnostic authority — this framing recurs throughout the interface and is treated as a hard constraint on scope, not a legal footnote.

---

## 2. Problem Statement & Scope

### 2.1 The problem

People experiencing symptoms face three disconnected decisions: what this might be, how serious it is, and what is usually done about it. Existing symptom checkers mostly answer only the first, and stop at a single guess rather than a ranked, evidenced set of possibilities. Chronic disease risk — whether a lifestyle profile elevates risk for conditions like diabetes or stroke — is usually a separate tool, disconnected from the acute symptom question. Severity triage is often folded into the same learned model that makes the diagnosis, which is risky: a model trained on incidental, imperfectly labelled data is a poor place to decide whether someone needs an ambulance. And treatment guidance is frequently sourced from a single evidence pool — often patient-reported satisfaction — without distinguishing that from what clinicians actually prescribe.

MedAssist AI treats these as four related but distinct problems and solves each with the component built for it, rather than asking one model to do all four jobs.

### 2.2 In scope

- Symptom-to-disease ranking across 684 disease classes, using structured yes/no symptom input.
- Chronic risk scoring for ten conditions from a lifestyle and demographic profile, each with its own calibrated probability and tuned decision threshold.
- A deterministic, rule-based severity/triage engine that combines diagnosis confidence, chronic risk, vital signs, age, symptom burden, and explicit red-flag symptoms into one of four bands: Mild, Moderate, Urgent, Emergency.
- Treatment recommendation for a resolved condition, drawing first on real hospital discharge prescribing data (MIMIC-IV) where the evidence supports it, and falling back to a popularity-and-quality-ranked view of patient drug reviews otherwise.
- Patient and provider-facing dashboards: symptom checker, risk assessment view, health reports, provider reports, prescriptions, and history.
- A PostgreSQL persistence layer recording assessments, patient and provider profiles, prescriptions, and provider-authored reports.

### 2.3 Out of scope

- Clinical diagnosis or prescription authority — every output is explicitly informational decision support, and the interface states this at the point of use.

- Imaging, lab-value interpretation, or any modality beyond structured symptoms, lifestyle survey data, vitals, and free-text diagnosis strings.
- Real-time emergency dispatch or direct integration with emergency services — the Emergency triage band instructs the user to seek care, it does not summon it.
- Drug interaction checking or dosage calculation — recommended drugs are named, not dosed.

**Design constraint** — Anything that decides whether a person needs urgent care is rule-based and clinician-defined, not learned, because no labelled triage dataset exists to validate a model against. This constraint shows up directly in the architecture in Section 3 and is treated as non-negotiable.

### 3. System Architecture

The system is organised in four layers: a client-facing application, an API layer, an inference layer that loads pre-trained model artifacts, and a persistence layer. Models are never trained inside the application — every model is trained offline in a notebook environment, exported as a versioned artifact, and loaded once at application startup. This separation keeps request latency low (each inference call is a handful of milliseconds once the artifacts are resident in memory) and keeps the training process reproducible and reviewable independently of the running application.

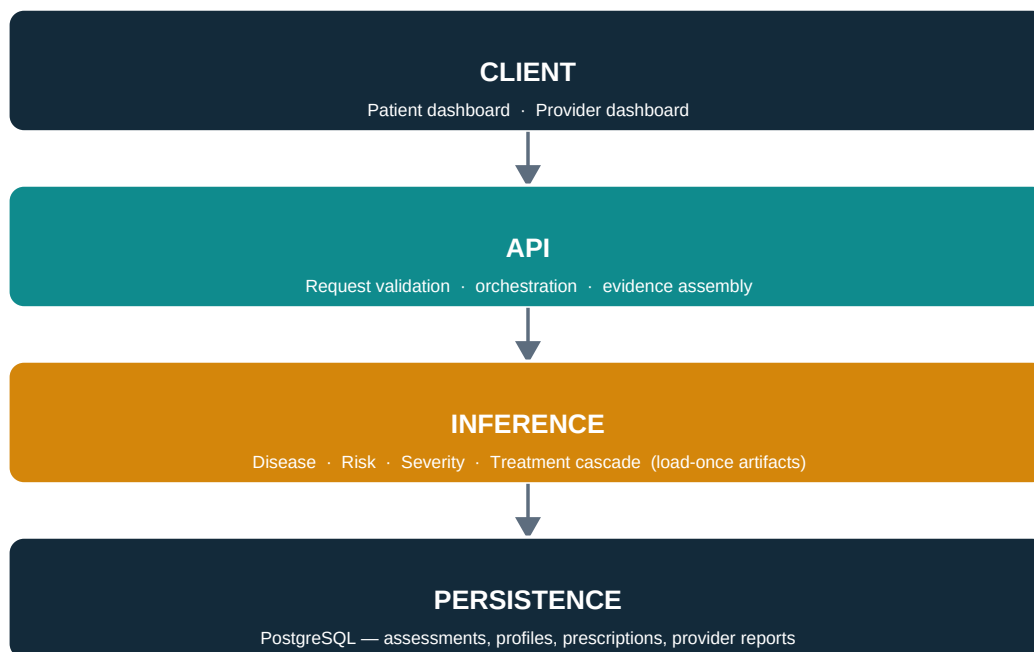


Figure 1. Four-layer system architecture — models load once at startup; nothing trains at request time.

#### 3.1 High-level data flow

A consultation moves through the pipeline as follows: symptoms are vectorised against a fixed 377-column schema and passed to the disease classifier, which returns ranked candidates. In parallel, the lifestyle profile passes to the ten chronic-risk models, each returning a calibrated probability and percentile. Both outputs, plus vitals, age, and any red-flag symptoms, feed the severity engine, which checks red-flag overrides first and only then computes a weighted score. The top disease candidate is separately resolved to a treatment recommendation via the two-layer cascade in Section 5. The assembled report — candidates, risk scores, severity band, and treatment recommendation, each with its own evidence and caveats — is persisted and rendered to the patient and, where applicable, the assigned provider.

Table 1. System architecture layers and responsibilities.

| Layer       | Responsibility                                                                                         | Key characteristic                                                          |
|-------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Client      | Symptom checker, risk assessment, reports, prescriptions, and history views for patients and providers | Role-aware; a single account surfaces a patient or provider dashboard       |
| API         | Request validation, orchestration across the four model components, response assembly                  | Stateless; every response carries the evidence behind each score            |
| Inference   | Loads pre-trained artifacts once at startup; answers disease, risk, severity, and treatment queries    | No training at request time; sub-10ms typical inference latency once loaded |
| Persistence | PostgreSQL: assessments, patient/provider profiles, prescriptions, provider reports                    | Every assessment is durable and auditable after the fact                    |

### 3.2 Why four components instead of one model

A single end-to-end model mapping symptoms directly to a treatment would be simpler to describe but far harder to trust: it would hide which part of the input drove which part of the output, make it impossible to hold the severity logic to a higher evidentiary standard than the diagnosis logic, and force every future improvement to risk destabilising the whole system. Decomposing the pipeline into disease prediction, risk scoring, severity triage, and treatment recommendation means each component can be evaluated against its own held-out data and its own baseline, and each can evolve — for example, the treatment layer moving from a single evidence source to a two-layer cascade — without touching the other three.

### 3.3 The treatment cascade, in outline

Section 5 covers this in depth, but it is worth stating here as an architectural decision: treatment recommendation is itself a two-layer system. A gate — tuned on held-out data, not hand-picked — decides per query whether real hospital discharge prescriptions (MIMIC-IV) are similar enough and well-supported enough to answer confidently. If they are, that evidence is returned. If not, the query falls through to a Bayesian-shrunk ranking over patient drug reviews. Every response names which layer answered and why, so the distinction between "clinicians prescribed this for comparable admissions" and "patients rated this well" is never collapsed into one undifferentiated recommendation.

---

## 4. Technology Stack & Modules

The stack separates concerns the same way the architecture does: a relational store for durable records, a Python-centred model and inference layer built on the standard scientific stack, and an application layer that composes the two into the dashboards described in Section 7.

**Table 2. Technology stack by architectural layer.**

| Layer              | Technology                                                                                            | Used for                                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Database           | PostgreSQL                                                                                            | assessments, patient_profiles, provider_profiles, prescriptions, provider_reports, users, system_settings   |
| Model training     | Python, scikit-learn, pandas, NumPy, SciPy                                                            | All four model components; trained and evaluated in notebook environments (Kaggle)                          |
| Model artifacts    | joblib (models), NumPy .npz (sparse matrices), JSON / CSV (metrics, lookup tables, configuration)     | Portable, version-controlled outputs loaded by the inference layer at startup                               |
| Text vectorisation | TF-IDF (scikit-learn), char n-gram and word n-gram variants                                           | Free-text diagnosis and review matching across Models 1, 3, and the MIMIC layer                             |
| Application / API  | A stateless service layer orchestrating the four model components and assembling per-request evidence | Symptom checker, risk assessment, treatment lookup endpoints                                                |
| Client             | Role-aware dashboard application (patient / provider views)                                           | Symptom Checker, Risk Assessment, My Reports, Provider Reports, Prescriptions, Complete History, My Profile |
| Severity logic     | severity_config.json — a hand-authored, clinician-reviewed rule set                                   | Red-flag overrides and the six-factor weighted severity score                                               |

*A note on this section:* the framework-level choices above (API framework, client framework) reflect the codebase as reviewed; if the current repository has since standardised on a different specific framework, that detail should be confirmed against the live package.json / requirements.txt before this document is treated as the source of truth for onboarding.

## 4.1 Core modules

- **Disease Prediction Service** — loads the Bernoulli Naive Bayes classifier, the symptom column schema, and the label encoder; accepts a symptom set and returns ranked candidates.
- **Risk Assessment Service** — loads ten calibrated gradient-boosted models and their tuned per-condition thresholds; accepts a lifestyle profile and returns a composite risk score, per-condition percentiles, and a risk band.
- **Severity Engine** — a pure rule evaluator reading severity\_config.json; combines the outputs of the other three services with vitals and red flags into a final triage level.
- **Treatment Cascade Service** — the two-layer recommender: a gated MIMIC-IV layer plus a drug-review fallback layer, described fully in Section 5.
- **Reporting & Persistence** — writes each assessment, its scores, and its treatment recommendation to PostgreSQL, and renders patient and provider report views from that record.

## 5. Dataset & AI/ML Models

Four datasets feed the pipeline, each mapped to the model it was chosen for. All models are evaluated on held-out data against an explicitly stated baseline — a discipline applied consistently across the project, because an accuracy figure means little without something dumb to compare it against.

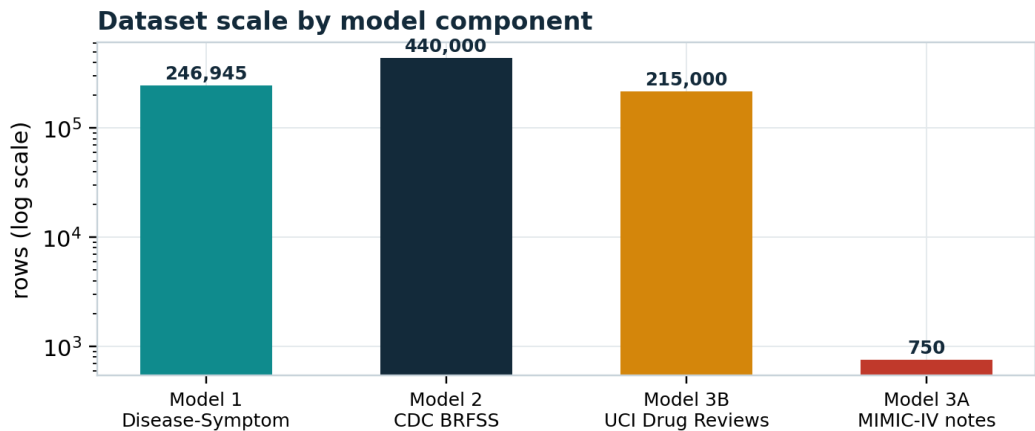


Figure 2. Dataset scale by model component, log scale — the MIMIC-IV layer is orders of magnitude smaller by design (see Section 5.4).

### 5.1 Model 1 — Disease Prediction

Table 3. Model 1 — Disease Prediction, dataset and configuration summary.

| Property         | Detail                                                                                                                                                                                         |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Datasets         | Disease-Symptom dataset (246,945 rows × 378 columns), merged with a 349-row real patient-profile dataset; a 99-row reference table supplies cure, doctor type, and risk level for display only |
| Schema           | 377 binary symptom columns (fever, cough, fatigue, difficulty_breathing, ...) plus one disease label; combined and de-duplicated down to 684 disease classes                                   |
| Algorithm        | Bernoulli Naive Bayes, selected against Complement NB, a closed-form softmax ridge classifier, and an ensemble                                                                                 |
| Selection metric | Top-3 accuracy — the interface presents a short ranked list for confirmation rather than a single forced diagnosis                                                                             |
| Result           | 87.04% top-1 accuracy, 96.50% top-3, 98.39% top-5, macro-F1 0.8659, against a 0.92% majority-class baseline                                                                                    |

Training normalises and merges disease name aliases, drops classes with too few rows, streams the sparse 247k × 377 matrix rather than densifying it, and computes the baseline before any model is fit. Bernoulli NB's independence assumption between symptoms is known to be false, but the errors it introduces largely cancel out for ranking purposes, which is what the top-3 selection metric actually rewards.

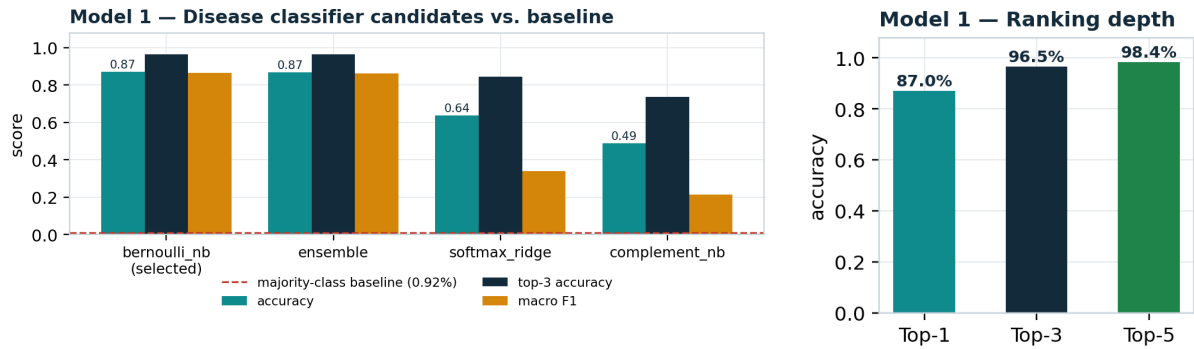


Figure 3. Model 1 candidate comparison against the majority-class baseline (left), and ranking depth of the selected model (right).

5.2 Model 2 — Chronic Disease Risk Assessment

Table 4. Model 2 — Chronic Disease Risk Assessment, dataset and configuration summary.

| Property         | Detail                                                                                                                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset          | CDC BRFSS (Behavioral Risk Factor Surveillance System), ~440,000 respondents                                                                                                                                          |
| Schema           | Coded survey features (BMI, smoking, exercise, alcohol, sleep, general health, age bracket, sex — coded 1/2/7/9 where 7/9 mean don't-know/refused and must be decoded before use) against ten binary condition labels |
| Algorithm        | Ten independent HistGradientBoostingClassifier models, each followed by isotonic regression calibration                                                                                                               |
| Threshold tuning | A per-condition decision threshold tuned individually — not a blanket 0.5 — because base rates range from 3.4% (kidney disease) to 34.4% (arthritis)                                                                  |
| Result           | Mean accuracy 0.8222, precision 0.2995, recall 0.5115, F1 0.3722, ROC-AUC 0.7967 across the ten conditions                                                                                                            |

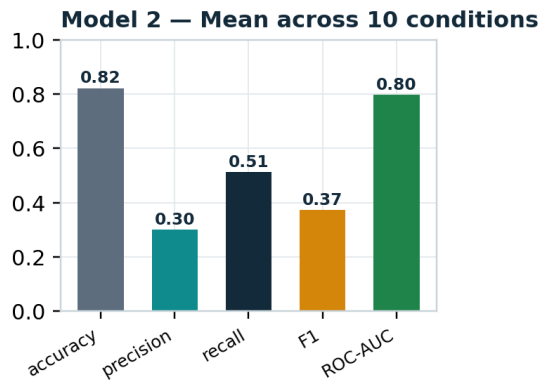


Figure 4. Model 2 mean scores across all ten conditions.

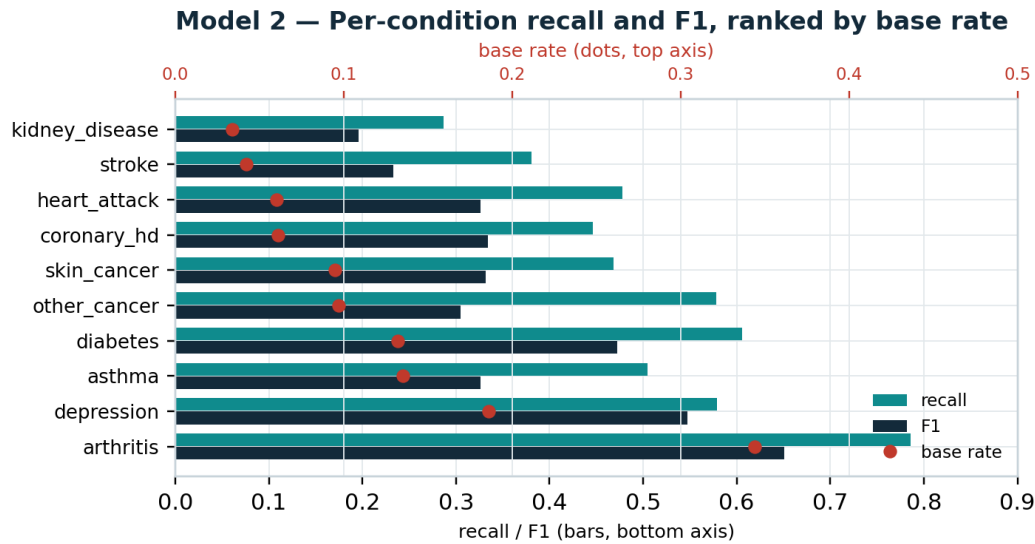


Figure 5. Recall and F1 per condition, ranked by base rate — recall consistently tracks how common the condition is, not model quality alone.

Eight of the ten conditions score lower on plain accuracy than a model that always predicts "nobody has it" — this is the deliberate outcome of prioritising recall over accuracy for a screening task with a low base rate, not a modelling failure. A screener that never flags anyone would score well on accuracy and catch nothing.

5.3 Severity / Triage Engine (rule-based, not learned)

Inputs — current symptoms and intensity, age, vitals, diagnosis confidence from Model 1, and chronic risk from Model 2 — first pass through a red-flag override gate. Any single critical red flag (sharp chest pain, vomiting blood, seizures, focal weakness, throat swelling) or two or more serious flags (shortness of breath, palpitations, fainting, dizziness) forces an immediate Emergency classification and skips all further scoring. Absent an override, six weighted factors combine into one 0-1 score, shown in Figure 6. The resulting score maps to Mild (0-0.25), Moderate (0.25-0.50), Urgent (0.50-0.75), or Emergency (0.75-1.0). All weights, red-flag lists, and vital-sign ranges are defined in severity\_config.json and were set by clinical review rather than learned from labelled data — no labelled triage dataset exists, and safety-critical escalation logic needs to be auditable, not a black box.

Severity score — six weighted factors

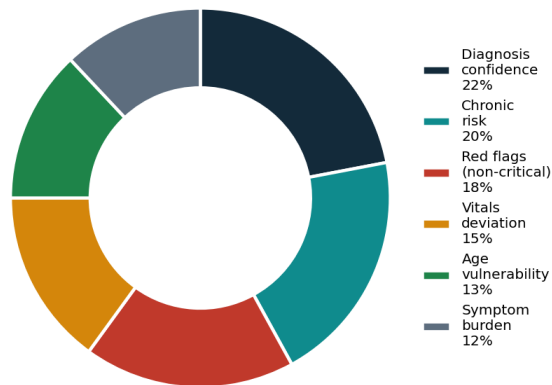
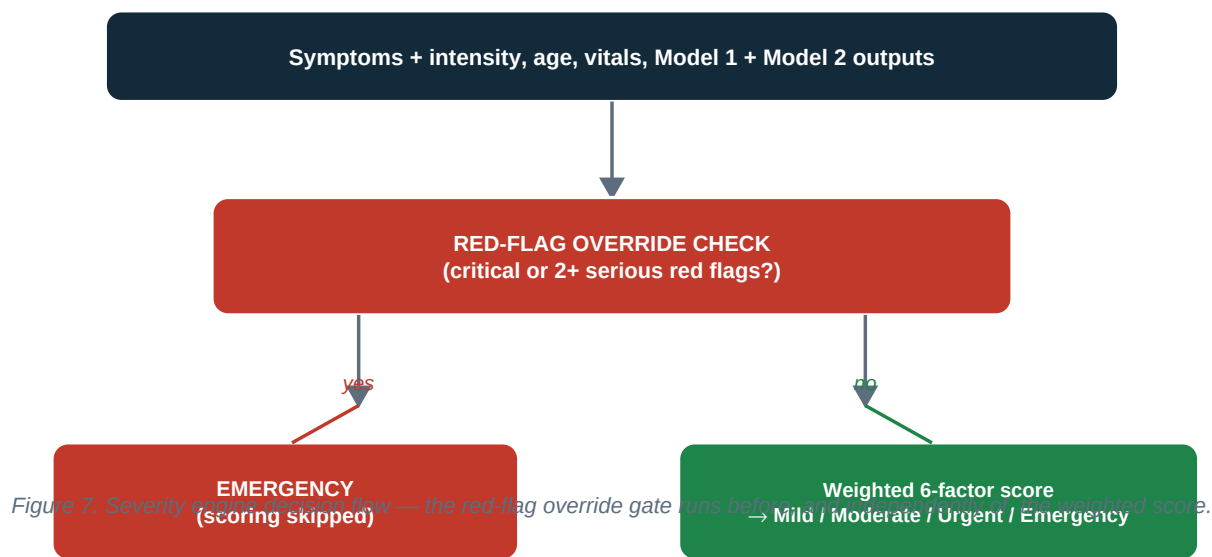


Figure 6. Composition of the six-factor weighted severity score.



### 5.4 Model 3 — Treatment Recommendation (two-layer cascade)

Treatment recommendation combines two datasets built for different jobs, ordered so the stronger evidence source answers first wherever it can support an answer.

Table 5. Model 3 — the two evidence layers behind treatment recommendation.

| Layer                           | Dataset & schema                                                                                                                                                                 | Algorithm                                                                                                                                                                                                                                    | Role                                                                                                                                                  |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| A — MIMIC-IV (preferred)        | 999 hospital discharge notes (~750 usable); free-text diagnosis and medications sections, regex-extracted, filtered of de-identification placeholders                            | Two-stage OneVsRest logistic regression over TF-IDF: Stage 1 predicts a drug class (12 classes) from the diagnosis text; Stage 2 predicts a specific drug within that class, with a frequency fallback where a class has under 15 admissions | Answers first, but only when a similarity-and-support gate — tuned on held-out admissions — is cleared. Confidence label: real hospital prescriptions |
| B — UCI Drug Reviews (fallback) | ~215,000 patient drug reviews (drugName, condition, rating 1-10, usefulCount, review text); a 329-class ComplementNB classifier separately maps free-text queries to a condition | Bayesian-shrunk rating (12 pseudo-counts) blended with review volume via a tuned popularity exponent (gamma), swept and selected on held-out reviews                                                                                         | Answers every query the MIMIC layer declines or has no support for. Confidence label: patient-reported experience                                     |

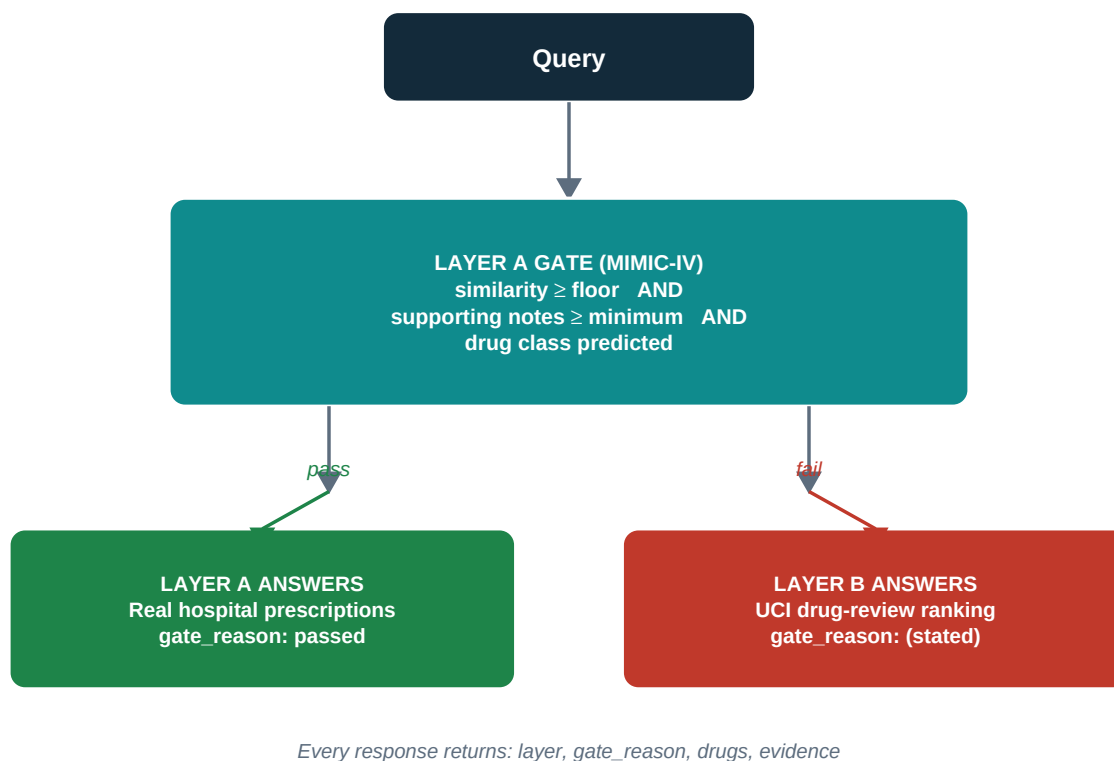


Figure 8. Treatment-cascade routing — Layer A answers only when its gate is cleared; every failure routes to Layer B with a stated reason.

**The gate.** Layer A does not answer every query just because it goes first — it is gated on three conditions read from the trained artifact: the best matching diagnosis must exceed a similarity floor, at least a minimum number of real notes must sit above that floor (so an answer never rests on a single admission), and the Stage 1 classifier must actually name a drug class. Any failure routes the query to Layer B, with the specific reason recorded (similarity\_below\_floor, insufficient\_support, no\_class\_predicted). The floor itself is swept across a grid and selected by measuring, on admissions held out of training, whether the resulting cascade actually beats the reviews-only baseline — not chosen by hand.

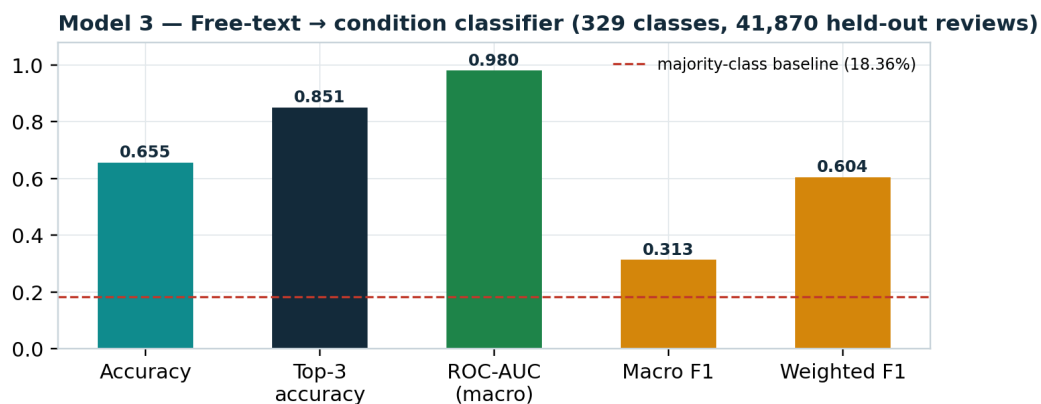


Figure 9. Free-text → condition classifier vs. the majority-class baseline.

**Table 6. Model 3 headline metrics and the current treatment-cascade gate configuration.**

| Component                                   | Metric                                   | Value                                          |
|---------------------------------------------|------------------------------------------|------------------------------------------------|
| Model 3 — text-to-condition classifier      | Accuracy / Top-3 accuracy                | 65.55% / 85.07% (329 classes, baseline 18.36%) |
| Model 3 — text-to-condition classifier      | ROC-AUC (macro, one-vs-rest)             | 0.9796                                         |
| Model 3 — text-to-condition classifier      | Macro F1 vs weighted F1                  | 0.3127 vs 0.6035                               |
| Treatment cascade — tuned gate (latest run) | Similarity floor / min. supporting notes | $\geq 0.10$ / $\geq 3$                         |

**Known caveat carried into Section 8** — The gate is tuned on held-out MIMIC diagnoses, which are themselves hospital-style phrasing — so the tuning measures routing quality on in-domain queries, not that hospital notes outperform patient reviews in general. In the most recent run the floor settled at the loosest value tested (0.10), which risks over-firing on queries far outside hospital medicine (dermatology, contraception). This is flagged explicitly as a retuning item in Section 8 and Section 11 rather than silently shipped.

Both layers carry an explicit epistemic caveat that is preserved into every API response: a discharge note lists every drug given to a patient who often had multiple concurrent problems, so a drug's presence next to a diagnosis is co-occurrence, not proof of attribution; and a review-site rating reflects patient-reported satisfaction, not clinical efficacy or safety. Neither layer is a prescription, and the interface never presents either one as such.

## 6. Implementation & Milestones

The project was built as a sequence of phases, each closing with a working, evaluated component before the next was layered on top. This kept every stage independently demonstrable rather than deferring all validation to a single final integration.

**Table 7. Development phases and current status.**

| Phase                                                        | Deliverable                                                                                                                                                                                 | Status                                                                             |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Phase 1 — Foundations                                        | Requirements, dataset acquisition and cleaning, PostgreSQL schema design (assessments, patient/provider profiles, prescriptions, provider_reports), initial application scaffolding         | Complete                                                                           |
| Phase 2 — Disease Prediction & Risk Assessment (Milestone 2) | Model 1 (symptom-to-disease, 684 classes) and Model 2 (ten chronic-risk models with calibration), plus the rule-based severity/triage engine and its dashboard surfaces                     | Complete — documented in the Milestone 2 deliverable                               |
| Phase 3 — Treatment Recommendation                           | Model 3 baseline: Bayesian-shrunk drug ranking over UCI reviews, plus the free-text condition classifier; disease-to-condition linking from Model 1's output                                | Complete                                                                           |
| Phase 4 — Two-Layer Treatment Cascade                        | MIMIC-IV discharge-prescription layer added in front of the review-based layer, with a similarity-and-support gate tuned on held-out admissions; artifact packaging and backend integration | In progress — gate retuning flagged for the next iteration (Section 8, Section 11) |

## 6.1 Training and export workflow

Every model is trained in a notebook environment (Kaggle), never inside the running application. Each notebook run ends by exporting a fixed set of artifacts — trained model objects (joblib), vectorised feature matrices (NumPy .npz), lookup tables and metrics (CSV / JSON) — and packaging them into a single versioned zip alongside a manifest recording file sizes, the tuned gate configuration, and a deployment checklist that fails loudly if a required file is missing. This keeps a hard boundary between training-time code (which fits models) and application-time code (which only loads and calls them), so a request can never trigger retraining and inference latency stays in the low milliseconds once artifacts are resident in memory.

## 6.2 Integration discipline

- Model thresholds and gate parameters (Model 2's per-condition cutoffs, the treatment cascade's similarity floor) are read from the artifact at load time, never hard-coded in application code, so training and serving cannot silently drift apart.
- Every model is fit twice where tuning is involved: once on a train split to select thresholds honestly against held-out data, and once on the full dataset for the artifact that ships. Tuning against data a model has already seen was treated as a correctness bug, not a shortcut.
- An unresolved or unmatched input (a disease with no linked treatment condition, a query neither evidence layer can support) returns an explicit empty result with a stated reason, rather than the nearest plausible-looking answer.

---

# 7. User Roles & Dashboard

The application is role-aware from a single account model: the same person can be a patient in one context and, if credentialed, a provider in another. The dashboard surfaces differ accordingly, but both are built from the same underlying assessment records.

## 7.1 Patient role

Table 8. Patient dashboard — screens and purpose.

| Screen           | Purpose                                                                                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dashboard        | Landing summary of recent assessments and headline health status                                                                                                                              |
| Symptom Checker  | A four-step flow — Symptoms, Health Profile, Vitals, Results — ending in a ranked disease list, a triage banner, and (when a condition resolves) a treatment recommendation with its evidence |
| Risk Assessment  | Composite risk score, chronic-risk percentiles per condition, triage level, and the count of conditions flagged out of ten screened                                                           |
| My Reports       | A running history of past assessments, each downloadable as a self-contained PDF report                                                                                                       |
| Prescriptions    | Prescriptions issued by a provider against this patient's record                                                                                                                              |
| Complete History | The full longitudinal assessment trail                                                                                                                                                        |
| My Profile       | Demographic and account details                                                                                                                                                               |

7.2 Provider role

Providers see the same assessment data through a clinical lens: a Provider Reports view listing patients by assessment, with fields for provider-authored insights, treatment suggestions, and health recommendations that are persisted alongside — not instead of — the system-generated recommendation. This keeps the machine-generated evidence and the clinician's own judgment as two distinct, separately auditable records rather than merging them into one undifferentiated note.

7.3 Design principle: evidence over verdicts

Across both roles, the interface consistently shows the evidence behind a number rather than the number alone: a risk score is shown with its percentile and the underlying condition model's calibration; a disease candidate is shown with the symptoms that matched it; a treatment recommendation states which layer answered — real hospital prescriptions or patient-reported experience — and why. This was treated as a usability requirement, not a cosmetic add-on: a bare score invites over-trust, while a score with its evidence invites the correct amount of trust.

8. Testing & Performance Evaluation

Every model component is evaluated on data it did not train on, against a stated baseline chosen to be genuinely hard to beat rather than a strawman. The headline figures are collected below; per-model detail is in Section 5.

**Table 9. Headline evaluation results across all four model components.**

| Model                               | Metric                                                         | Result                                                                       | Baseline                                       |
|-------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------|
| 1 — Disease Prediction              | Top-3 accuracy                                                 | 96.50%                                                                       | 0.92%<br>(majority class)                      |
| 1 — Disease Prediction              | Top-1 accuracy / macro-F1                                      | 87.04% / 0.8659                                                              | —                                              |
| 2 — Chronic Risk (mean of 10)       | ROC-AUC / recall                                               | 0.7967 / 0.5115                                                              | Accuracy alone is misleading — see Section 5.2 |
| 3 — Text-to-condition classifier    | Top-3 accuracy / ROC-AUC                                       | 85.07% / 0.9796                                                              | 18.36%<br>(majority class)                     |
| 3 — Drug ranking (Layer B)          | Held-out hit rate vs. popularity-only ranking                  | Beats both a random and a popularity-only baseline (gamma tuned empirically) | —                                              |
| 3 — Treatment cascade (Layer A + B) | Held-out drug-recommendation hit rate vs. reviews-only routing | Cascade improves on reviews-only on in-domain (hospital-style) queries       | Reviews-only (current shipped behaviour)       |

## 8.1 Testing methodology

- **Held-out splits throughout.** Every reported metric is computed on data excluded from training, never on training data — including the treatment cascade's gate, which is tuned on a train split and only then evaluated on the held-out remainder.
- **Stated baselines, not implicit ones.** Each model is compared against the simplest competitor that could plausibly be shipped instead — majority-class prediction, popularity-only ranking, reviews-only routing — so an improvement is measured, not assumed.
- **Calibration checked, not just discrimination.** Model 2's isotonic calibration step is validated by comparing predicted probability bands against observed frequency in those bands, because a well-ranking but poorly-calibrated model can still mislead a user reading a specific percentage.
- **Threshold selection is metric-aware, not defaulted.** Model 1 selects on top-3 accuracy because the product surfaces a ranked list; Model 2 tunes a threshold per condition because base rates vary by an order of magnitude across the ten conditions.
- **Known limitation, tracked rather than hidden.** The treatment cascade's current gate (similarity  $\geq 0.10$ ) was tuned entirely against hospital-style diagnosis phrasing and has not yet been validated against clearly out-of-domain queries (e.g. dermatology, contraception). This is an open item carried into Section 11, not a resolved result.

## 8.2 Severity engine verification

Because the severity engine is rule-based rather than learned, its verification is different in kind: rather than a held-out accuracy figure, it is checked by scenario walk-through — confirming that each listed critical and serious red flag correctly forces an Emergency classification and bypasses weighted scoring, and that the six-factor

weighted score produces the expected band boundaries at 0.25, 0.50, and 0.75. This is treated as a correctness requirement equivalent in importance to a model's held-out metric, precisely because it governs the one output where an error carries direct safety consequences.

## 9. Deployment & Results

### 9.1 Artifact packaging

Each notebook run concludes by exporting every model artifact into a single directory and zipping it with a manifest, so a deployment is always a known, complete, checked bundle rather than a set of files copied by hand. The manifest records file sizes, a timestamp, whether the optional MIMIC-IV layer is present, and — critically — the tuned gate configuration for the treatment cascade, so the serving layer can read thresholds from the artifact rather than a value hard-coded separately in the backend.

Table 10. Deployment artifact groups and their requirement status.

| Artifact group              | Files                                                                               | Required for serving                                                               |
|-----------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Model 1                     | classifier, label encoder, symptom column schema, symptom evidence, metrics         | Yes                                                                                |
| Model 2                     | ten risk models bundle, per-condition metrics, overall metrics                      | Yes                                                                                |
| Model 3 — Layer B (reviews) | treatment table, disease-condition link, condition classifier, metrics              | Yes                                                                                |
| Model 3 — Layer A (MIMIC)   | two-stage model bundle (with tuned gate), diagnosis matrix, source records, metrics | Optional — serving degrades to Layer B only if absent, and reports this explicitly |
| Severity                    | severity_config.json (weights, red-flag lists, vital ranges)                        | Yes                                                                                |

### 9.2 Deployment behaviour

- Model artifacts load once at application startup; a startup health check logs which models loaded, whether the MIMIC layer is active, the active gate thresholds, and row counts for each dataset, so a missing or stale artifact is visible in logs immediately rather than discovered from a user complaint.
- The 100MB free-text condition classifier is loaded lazily, on first use, rather than at startup, so it does not inflate baseline memory footprint for requests that never need it.
- Every treatment recommendation persists its source layer and routing reason to the assessments table, so production monitoring can track how often the MIMIC layer actually fires versus falling through to reviews — a number the team does not yet have at scale and considers essential before widening the gate further.
- An unmatched query returns an explicit empty result with a stated reason rather than the nearest plausible-looking condition, treated throughout as correct behaviour rather than a gap to be silently filled.

## 9.3 Results to date

Phases 1 through 3 are complete and validated against held-out data as summarised in Section 8. Phase 4 — the two-layer treatment cascade — is functionally integrated and packaged for deployment, with one open finding carried forward rather than resolved: the current gate threshold was tuned exclusively against hospital-style diagnosis phrasing, and the resulting floor (similarity  $\geq 0.10$ ) is the most permissive value tested. Before this is treated as production-ready, it needs re-tuning against a query set that includes conditions genuinely absent from the MIMIC-IV corpus (dermatology, contraception, common ambulatory complaints), so the gate is scored on its actual failure mode rather than only on the domain it already knows.

---

## 10. Conclusion

MedAssist AI shows that a symptom-to-treatment pipeline can be built as independently evaluated, purpose-built components rather than one opaque model, without losing the coherence of a single workflow. Each stage uses the technique suited to its evidence — Naive Bayes for ranking over sparse symptom data, calibrated gradient boosting for imbalanced chronic-risk screening, explicit clinician-authored rules for the one decision where an unlabelled model would be unsafe, and a gated two-layer cascade that lets stronger clinical evidence take precedence over weaker patient-reported evidence wherever it can.

The project's most consistent discipline is measuring every component against a baseline chosen to be hard to beat, and stating plainly where a result does not yet clear it. The treatment cascade's gate is the clearest example: rather than shipping a plausible-looking threshold, the notebook measures whether the cascade actually beats reviews-only routing on held-out data — and the finding that the tuned floor may be too permissive outside hospital-style queries is documented as an open item, not smoothed over. That habit is what makes the stronger results (96.5% top-3 disease accuracy against a 0.92% baseline; 85% top-3 condition match against an 18% baseline) credible rather than merely reported.

---

## 11. Future Enhancements

### 11.1 Near-term (next iteration)

- **Re-tune the treatment cascade's gate against out-of-domain queries.** Add UCI conditions known to be absent from MIMIC-IV (dermatology, contraception, routine ambulatory complaints) as explicit negatives in the sweep, so the floor is selected against its actual failure mode rather than only against hospital-style phrasing.
- **Production monitoring for cascade routing.** Track the persisted layer / gate\_reason fields at scale to learn, empirically, how often MIMIC evidence fires and where it fails to, before considering any further loosening of the gate.
- **Expand the MIMIC-IV extract.** The current layer trains on roughly 750 usable admissions; a larger extract would let more of the twelve drug classes graduate from a frequency fallback to a trained Stage 2 model.

### 11.2 Medium-term

- **Drug interaction and contraindication checking** layered on top of the treatment cascade's recommendations, explicitly flagged as a separate safety feature from the recommendation itself.
- **Provider-side feedback loop** — capturing whether a provider accepted, modified, or rejected a system-generated treatment recommendation, as a labelled signal for future retraining that today's pipeline does not yet have.
- **Longitudinal risk tracking** for the chronic-risk models, so a patient's risk trend over repeated assessments is surfaced, not just a single point-in-time score.
- **Multilingual symptom input** for the free-text condition classifier, which currently assumes English-language review and diagnosis text.

### 11.3 Longer-term

- Extending the two-layer evidence pattern established for treatment recommendation to disease prediction itself — for example, gating a future clinical-notes-based diagnosis layer in front of the current symptom-survey-based classifier, the same way MIMIC-IV now sits in front of drug reviews.
- A structured red-flag and vitals capture flow that reduces reliance on free-text symptom entry, improving the reliability of the severity engine's inputs at the source.
- Formal clinical validation of the severity engine's weights and thresholds against real triage outcomes, should a suitably labelled dataset become available — at which point the engine's current hand-authored weights would become a baseline to beat, not a fixed design.

---

*This document reflects the MedAssist AI pipeline as implemented through the two-layer treatment cascade (Phase 4), prepared by **Dharani K**. It is intended as a living reference — sections describing in-progress or open items (Sections 5.4, 8.1, 9.3, 11) should be revisited and updated as the corresponding retuning and validation work is completed.*